
Übungsblatt 5

Abgabe: 26.05.2026, 10:00 Uhr (im Digicampus via VIPS: .java-Dateien für Code, .uxf für UML, .pdf für alles andere)

- Dieses Übungsblatt muss im Team abgegeben werden (Einzelabgaben sind nicht erlaubt!).
- Die **Zeitangaben** geben zur Orientierung an, wie viel Zeit für eine Aufgabe später in der Klausur vorgesehen wäre; gehen Sie davon aus, dass Sie zum jetzigen Zeitpunkt wesentlich länger brauchen und die angegebene Zeit erst nach ausreichender Übung erreichen.

* leichte Aufgabe / ** mittelschwere Aufgabe / *** schwere Aufgabe

Aufgabe 17 *** (*Polymorphismus / Schnittstellen, 20 Minuten*)

a) (***, 10 Minuten)

Erklären Sie die Begriffe **Substitutionsprinzip**, **spätes Binden** und **Polymorphismus** anhand des Eingabeparameters und der Implementierung der `indexOf(Object o)`-Methode der Klasse `ArrayList`.

b) (***, 10 Minuten) Erklären Sie die Vorteile der Benutzung von **Schnittstellen als Datentypen** gegenüber konkreten Klassen anhand des Eingabeparameters der `addAll`-Methode der Klasse `ArrayList`.

Aufgabe 18 (*Collection-API, 20 Minuten*)

a) (**, 10 Minuten)

Schreiben Sie ein übersetzbare Java-Programm, das

- ein streng typisiertes `ArrayList`-Objekt `myList` für `String`-Objekte erzeugt,
- die drei Strings `"Mein"`, `"kleiner"`, `"grüner"` zu `myList` hinzufügt,
- die Kommandozeilenparameter **von vorne nach hinten** durchläuft und so zu `myList` hinzufügt, dass sie am Ende in umgekehrter Reihenfolge in `myList` enthalten sind. Werden also zum Beispiel die Kommandozeilenparameter `"Balkon"`, `"am"`, `"draußen"`, `"steht"`, `"Kaktus"` übergeben, sollen sie in `myList` in der Reihenfolge `"Kaktus"`, `"steht"`, `"draußen"`, `"am"`, `"Balkon"` enthalten sein,
- für jeden der Strings `"Hollari"`, `"hollari"`, `"hollaro"` in der angegebenen Reihenfolge überprüft, ob sie in `myList` enthalten ist, und falls das nicht der Fall ist, sie zu `myList` hinzufügt und
- die letzten 5 Strings unter Benutzung eines Iterators (Hinweis: `listIterator`-Methode) und seinen `hasNext` und `next`-Methoden getrennt durch Kommata in einer Zeile ausgibt.

b) (****, 10 Minuten)

Schreiben Sie ein übersetzbares Java-Programm, das

- ein streng typisiertes `HashSet`-Objekt `mySet` für `Character`-Objekte erzeugt,
- für jeden übergebenen Kommandozeilenparameter alle enthaltenen Zeichen zu `mySet` hinzufügt,
- `mySet` ausgibt und
- für jedes Element aus `mySet` die Kommandozeilenparameter ausgibt, in denen das entsprechende Zeichen vorhanden ist. Dazu soll
 - pro Zeichen in einer Zeile das Zeichen selbst ausgegeben werden,
 - für dieses Zeichen eine Liste aller Kommandozeilenparameter bestimmt werden, in denen das Zeichen vorkommt, und
 - diese Liste in einer neuen Zeile durch Kommata getrennt ausgegeben werden. Verwenden Sie dafür **nicht** die String-Darstellung der Liste, sondern erzeugen Sie die Darstellung mit der `join`-Methode der Klasse `String`.

Beispiel: Für die Kommandozeilenparameter `"ab"`, `"ac"`, `"bc"` soll die Ausgabe wie folgt aussehen:

```
[a, b, c]
a
ab, ac
b
ab, bc
c
ac, bc
```

c) (***, 2 Minuten)

Modifizieren Sie das Programm aus b) so, dass für die Ausgabe von `mySet` der Inhalt erst in ein Array überführt wird. Sorgen Sie dafür, dass die Ausgabe genauso aussieht wie die direkte Ausgabe von `mySet`.

Hinweis: Nutzen Sie dazu die Methode `toArray()` und das späte Binden der `toString()`-Methode.

Aufgabe 19 **** (Unveränderliche Klassen und Containerklassen, 25 Minuten)

In dieser Aufgabe sollen Sie eine Datenklasse `Measurement` sowie eine zugehörige Containerklasse entwerfen und implementieren. Eine Messung besteht aus einem Zeitpunkt mit Datum und Uhrzeit, einer Sensor-Bezeichnung (z.B. `"Sensor-A1"`) sowie einem reellwertigen Messwert. Es sollen Methoden zur Verfügung gestellt werden, um diese Informationen auszulesen. Sorgen Sie dafür, dass die Klasse `Measurement` unveränderlich sind. Es soll eine Klassenmethode `of` zur Verfügung gestellt werden, um `Measurement`-Objekte erzeugen zu können.

a) (***, Klassenkarten entwerfen, 8 Minuten)

Entwerfen Sie Klassenkarten für die Klasse `Measurement` und die Containerklasse `MeasurementContainer`. Die Klasse `MeasurementContainer` soll die Schnittstelle `Iterable` implementieren. Orientieren Sie sich beim Entwurf der Containerklasse an Abbildung 4 in Kapitel 5 des Skripts.

b) (****, Klasse implementieren, 8 Minuten)

Implementieren Sie die Klasse `Measurement` so, dass sie unveränderlich ist. Denken Sie an den Konstruktor, passende Datentypen sowie eine Klassenmethode `of` zum Erzeugen neuer Objekte.

c) (**, Containerklasse implementieren, 8 Minuten)

Implementieren Sie die Containerklasse **MeasurementContainer**. Die Klasse soll intern eine Liste von **Measurement**-Objekten verwalten. Verwenden Sie dazu eine geeignete Collection. Stellen Sie eine Methode **addMeasurement** bereit, um eine neue Messung zur Messreihe hinzuzufügen, sowie eine Methode **removeMeasurement** zum Entfernen. Implementieren Sie außerdem die Schnittstelle **Iterable<Measurement>**.

*Hinweis: Bei der Implementierung der Verwaltungsmethoden reichen Sie die Eingabeparameter an das intern verwendete **Collection**-Objekt durch.*

d) (*, Programmklasse testen, 1 Minuten)

Testen Sie Ihre Implementierung mit der vorgegebenen Programmklasse **MeasurementTest**. Sie finden die Klasse im Digicampus, wo sich auch diese Angabe befindet.

Aufgabe 20 (Funktionale Schnittstellen und Lambda-Ausdrücke, 22 Minuten)

a) (**, 9 Minuten)

Schreiben Sie ein übersetzbares Java-Programm, das

- eine Liste für **String**-Objekte erzeugt und diese mit den Kommandozeilenparametern befüllt (verwenden Sie dafür eine geeignete Methode der Klasse **Arrays** und übergeben Sie deren Rückgabewert an einen Konstruktor z.B. eines neuen **ArrayList**-Objekts),
- ein **UnaryOperator**-Objekt für **Strings** erstellt, dass alle Kleinbuchstaben eines übergebenen Strings zu Großbuchstaben konvertiert,
- damit alle Strings in der vorher erstellten Liste transformiert,
- mit einem Lambda-Ausdruck für ein **Predicate<String>**-Objekt alle Strings, die mit **"A"** beginnen, entfernt und
- am Ende alle Strings jeweils in einer eigenen Zeile mithilfe einer Methodenreferenz ausgibt.

b) (***, 13 Minuten)

Schreiben Sie ein übersetzbares Java-Programm, das

- eine Liste für **Integer**-Objekte erzeugt und diese mit 5000 zufälligen ganzen Zahlen aus dem Intervall $[0, 10000[$ befüllt,
- direkt mit einem Lambda-Ausdruck alle geraden Zahlen der Liste durch ihren quadrierten Wert ersetzt,
- ein Objekt vom Typ **Predicate<Integer>** erzeugt, das zurückgibt, ob eine übergebene natürliche Zahl eine Primzahl ist,
- mit diesem Objekt alle Primzahlen aus der Liste entfernt und
- am Ende alle Zahlen in einer jeweils eigenen Zeile mithilfe einer Methodenreferenz ausgibt.